

Durum	Biten
Başladı	Cuma, 27 Mart 2026, 9:55 AM
Tamamlandı	Cuma, 27 Mart 2026, 10:18 AM
Süre	23 dk 32 sn
Not	55,00 üzerinden 34,50 (62,73%)

Soru 1

Yanlış

5,00 üzerinden 0,00 notunu ver

When you determined the version of the exam you should take, what was the total value of sum?

The 5 points bonus will be given for taking the correct version and a correct answer to this question.

```
if(correctVersion && thisQuestionCorrect)
```

```
ExamScore += 5;
```

Cevap: 36



Soru 2

Doğru

1,00 üzerinden 1,00 notunu ver

Which of the following is **not true** of abstract methods?

- ☒ a. The code inside of an abstract method must use abstract variables ✓
- ☐ b. They can exist in abstract classes
- ☐ c. All methods in an interface must be abstract or declared default
- ☐ d. A non-abstract child of the class must implement any abstract method in its parent class
- ☐ e. All of these are true

Soru 3

Doğru

5,00 üzerinden 5,00 notunu ver

Match the Java keyword with its **most common use**.

Used to call a method inside the same class.

this ⇅ ✓

Used to access a variable inside the same class

this ⇅ ✓

Used to call a constructor of a parent class

super ⇅ ✓

Used to call a method or variable of a parent class

super ⇅ ✓

Used to process an Exception (after it occurs)

catch ⇅ ✓

Used to declare a parent/child relationship between classes.

extends ⇅ ✓

Soru 4

Doğru

1,00 üzerinden 1,00 notunu ver

Which of the following is correct?

- ☐ a. An *object* is like the blueprint; we can have many instances of it which are called *classes*
- ☒ b. A *class* is like the blueprint; we can have many instances of it called which are called *objects* ✓
- ☐ c. There is no difference between a *class* and an *object*
- ☐ d. The **object** keyword indicates we're defining an *object* and the **class** keyword indicates we're defining a *class*

Soru 5

Doğru

1,00 üzerinden 1,00 notunu ver

Select all statements which are **true**

- ☒ a. Two methods in a class can have the same name. ✓
- ☐ b. Two methods in a class must have different names. Otherwise, we will have a compiler error.
- ☐ c. None of the other statements are true

Soru 6

Kismen doğru

1,00 üzerinden 0,50 notunu ver

Select all statements which are **false**

- ☐ a. All classes always have a no-argument constructor even if a constructor is not defined.
- ☒ b. A constructor must always be explicitly defined. ✓
- ☐ c. None of the statements are false

Soru 7

Doğru

1,00 üzerinden 1,00 notunu ver

Select all statements that are **true**

- ☒ a. Non-abstract classes **cannot** have abstract methods. ✓
- ☒ b. Abstract classes **can** have abstract methods. ✓
- ☒ c. Abstract classes **can** have non-abstract methods. ✓
- ☐ d. Abstract classes **must** have abstract methods.
- ☐ e. Non-Abstract classes **must** have non-abstract methods.
- ☐ f. None of these statements are true

Soru 8

Doğru

1,00 üzerinden 1,00 notunu ver

If a variable is declared as private in a class, how can its value be changed?

- ☐ a. Its value cannot be changed after the object is created since it is immutable.
- ☒ b. The value can be changed through a public method. ✓
- ☐ c. The value can be changed through direct access.
- ☐ d. The value can only be changed by an object of a parent type.
- ☐ e. The value can only be changed by an object of a child type.

Soru 9

Doğru

1,00 üzerinden 1,00 notunu ver

One difference between a static method and a non-static method is that a non-static method must be called using an object of the class type. A static method can be called without creating an object.

- ☒ Doğru ✓
- ☐ Yanlış

Soru 10

Doğru

1,00 üzerinden 1,00 notunu ver

An Exception must be caught inside the same method in which it was thrown.

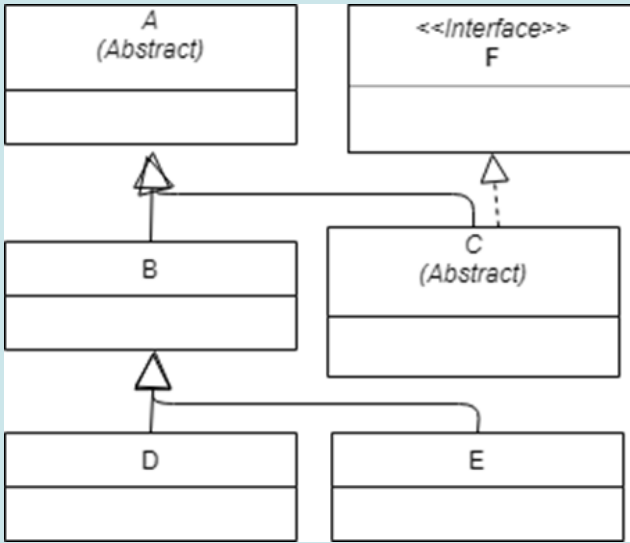
- ☐ Doğru
- ☒ Yanlış ✓

Soru 11

Doğru

5,00 üzerinden 5,00 notunu ver

Using the following UML class diagram select all statements that evaluate to **true**



- ☒ a. B instanceof A ✓
- ☐ b. D instanceof F
- ☐ c. F instanceof C
- ☒ d. E instanceof A ✓
- ☒ e. E instanceof B ✓

Soru 12

Tamamlandı

10,00 üzerinden 8,00 notunu ver

What is the output of the following?

```
public static void main(String[] args){
    try{
        int[] arr = {1, 2, 3, 4, 5};
        displayValue(arr, arr.length / 2);
        displayValue(arr, arr.length);
        displayValue(arr, 0);
    } catch (Exception ex){
        System.out.println("Error: In main method.");
    }
}

public static void displayValue(int[] arr, int i) throws IndexOutOfBoundsException {
    try {
        if(i >= arr.length || i < 0)
            throw new IndexOutOfBoundsException();
        System.out.println(arr[i]);
    } catch (IndexOutOfBoundsException ex){
        System.out.println("Error: index invalid");
        throw ex;
    } finally {
        System.out.println("finally");
    }
}
```

2

finally

Error: index invalid

finally

Error: In main method.

Soru bilgi

Analyze the code segments below. There is an error in each code segment. Write the line number where the error occurs. Select what type of error it is (Compiler, Runtime, Logical). State why the error occurs and give one possible way the code could be changed to remove the error.

Soru 13

Yanlış

2,00 üzerinden 0,00 notunu ver

```
1 public class Vize {
2     public static void main(String[] args) {
3         A a = new B();
4         a.print();
5     }
6 }
7
8 class A {
9     private String s;
10
11     public A(String newS) {
12         s = newS;
13     }
14
15     public void print() {
16         System.out.println(s);
17     }
18 }
19
20 class B extends A {
21 }
```

Write only the line number of where the error occurs.

Cevap: 3

**Soru 14**

Doğru

2,00 üzerinden 2,00 notunu ver

This is a Compiler error.**Soru 15**

Yanlış

2,00 üzerinden 0,00 notunu ver

Explain the cause of the error.

Cevap: we cannot initialize new object like A a= new B(); because b is child class of aa ar

**Soru 16**

Yanlış

3,00 üzerinden 0,00 notunu ver

Give one possible change to the given code that would remove the error. Include the line number and what the new line should be.

Cevap: B a= new A();



Soru 17

Cevaplanmadı

2,00 üzerinden puanlanmış

Suggest another change on a different line from your response in the previous question that would remove the error.

Cevap:

**Soru 18**

Doğru

2,00 üzerinden 2,00 notunu ver

Write only the line number of where the error occurs.

```
1 public class Midterm {
2     public static void main(String[] args) {
3         Parent p = new Child();
4         p.myMethod(3);
5     }
6 }
7
8 abstract class Parent {
9     public abstract void myMethod(int myInt) {
10         System.out.println("Parent myInt is " + myInt);
11     }
12 }
13
14 class Child extends Parent {
15     public void myMethod(int myInt) {
16         System.out.println("Child myInt is " + myInt);
17     }
18 }
```

Cevap: 10

**Soru 19**

Yanlış

2,00 üzerinden 0,00 notunu ver

This is a Logical error.

Soru 20

Yanlış

2,00 üzerinden 0,00 notunu ver

Explain the cause of the error.

Cevap: we cannot initialize anything in abstract method



Soru 21

Doğru

3,00 üzerinden 3,00 notunu ver

Give one possible change to the given code that would remove the error. Include the line number and what the new line should be.

Cevap: **Soru 22**

Doğru

2,00 üzerinden 2,00 notunu ver

Suggest another change on a different line from your response in the previous question that would remove the error. If the change you suggested in the previous question is the only possible change, write "None" for this question.

Cevap: 

AIRPORT GATE MANAGER SYSTEM

Please read this manual carefully before beginning your implementation in the Moodle VPL environment. You are expected to build several classes that interact with each other to manage flights at an airport terminal.

System Architecture and Class Overview

Your system will model different types of flights and an airport to manage them. You will use an inheritance hierarchy where a general flight class shares its properties with more specific flight types.

Deliverables

Submit a single Java file named `AirportManager.java` via LMS System (Moodle).

1. Class: Flight (Base Class) - abstract

This is the parent class representing a generic flight.

- Attributes:

- An integer representing the flight code.
- An integer representing the minutes remaining until departure.

- Constructor:

- Takes two parameters: `int flightCode` and `int minutesToDeparture` to initialize the attributes. (Note: These values can initially be 0, or even logically negative if a flight is already considered departed).

- Methods:

- Implement standard getter and setter methods for both attributes.
- `getFlightCode()` returning an `int`.
- `setFlightCode(int flightCode)` returning `void`.
- `getMinutesToDeparture()` returning an `int`.
- `setMinutesToDeparture(int minutesToDeparture)` returning `void`.

2. Class: DomesticFlight (Child Class)

This class extends the Flight base class.

- Attributes:

- An integer representing the terminal zone.

- Constructor:

- Takes three parameters: `int flightCode`, `int minutesToDeparture`, and `int terminalZone`. You must use the `super()` keyword to initialize the inherited attributes.

- Methods:

- `getTerminalZone()` returning an `int`.

3. Class: InternationalFlight (Child Class)

This class also extends the Flight base class.

- Attributes:

- An integer representing the time required for customs clearance (`customsTime`).

(Note: This value can be exactly 0).

- Constructor:

- Takes three parameters: `int flightCode`, `int minutesToDeparture`, and `int customsTime`. Again, utilize the `super()` keyword for the parent attributes.

- Methods:

- `getTotalMinutesToDeparture()` returning an `int`. This method calculates and returns the total time before the plane actually leaves.
- Crucial Rule: Do not calculate and store this total time in the constructor! You must dynamically calculate it (by adding the inherited `minutesToDeparture` to the `customsTime`) every time this method is called. This ensures the total time remains accurate even if the inherited minutes are changed later via a setter method.

The Manager Class: Airport

This class is the core engine of your simulation. It manages two separate fixed-size arrays representing the physical gates at the airport: one for domestic flights and one for international flights.

- Constructor:

- Takes two parameters: `int domesticCapacity` and `int intlCapacity`. It initializes the internal arrays using these capacities. (Note: The system should not crash if an airport is built with a capacity of 0!).

- Methods:

- `assignGate(DomesticFlight df)` returning `void`: This is an overloaded method. It iterates through the domestic gates array from the beginning (index 0). It finds the first empty spot (a null index), assigns the provided flight object to it. If the array is completely full, it throws the provided custom `DomesticFullException` with the index of the gate holding the flight scheduled to leave the soonest (the minimum minutes to departure).

- `assignGate(InternationalFlight itf)` returning `void`: This is the second version of the overloaded method. It performs the exact same logic as above, but operates exclusively on the international gates array and uses the `InternationalFullException`.

- `updateTime(int minutesPassed)` returning `void`:

This simulates the passage of time or delays. It iterates through both arrays. For every active flight, it subtracts the `minutesPassed` from the flight's current minutes to departure. Note: if `minutesPassed` is negative, that is how we will simulate a delay to all planes.

- Departure Condition: If a flight's remaining time drops to exactly 0 or below, it has departed! You must free up the gate by setting that array index back to null.

- Safety Check: Make sure your loops safely skip over already empty (null) gates without throwing a `NullPointerException`.

NOTE: the provided Exception classes should be children of the java Exception class as shown in the example UML diagram below describing the system.

